Copper: from a sentence to a manufacturable PCB

An AI agent that takes a plain-English hardware requirement and produces a checked, ready-to-manufacture circuit board, and fixes its own mistakes using neatlogs traces. Proposal written 10 October 2026, during the hackathon. Nothing here is built or measured yet; it reuses infrastructure we built and measured today.

> **High risk, high reward: we are betting on a task most teams will avoid, because it is hard, visual and objectively checkable. If it works, it will stand out from the rest of the field.**

1. The challenge

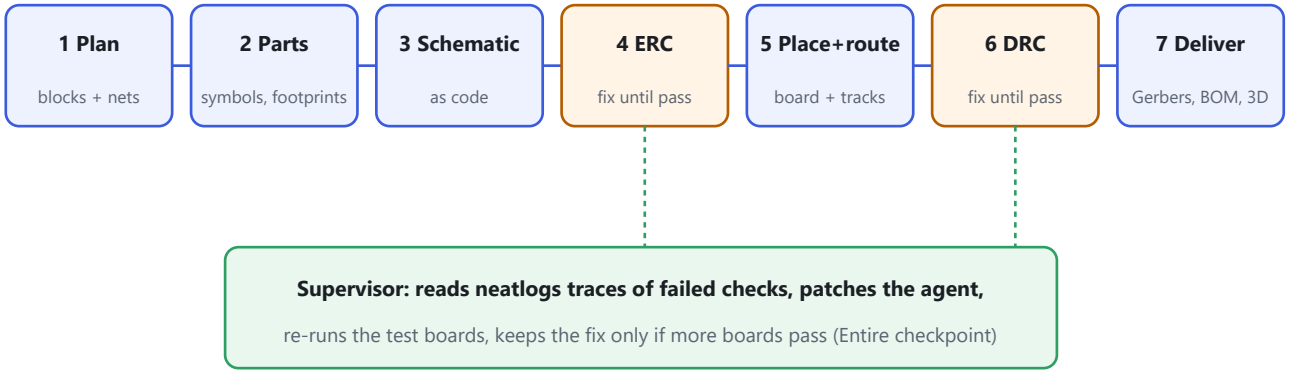
neatHack's only track: **build an agent that completes a substantial task from start to finish**, covering planning, tool use, state, recovery, verification and iteration. Every project must use **neatlogs**, **Entire** and **cfo.ai**. Judging: the agent 30%, the three tools 25%, demo video 20%, building in public 15%, usefulness and originality 10%. Submission: Monday 12 October, 7:00–11:55 PM IST, on X.

2. The problem

Going from "I need a board that does X" to a correct schematic and PCB takes an engineer days: choosing parts, wiring the schematic, placing parts, routing copper, then fixing checker errors one by one. Hobbyists, students and small hardware teams get stuck at exactly this step. A chat assistant can describe a circuit, but it cannot hand over files a factory will accept, and it cannot prove they are correct.

3. Our solution

The user types one requirement, for example: "A USB-C powered board with an ESP32, a temperature sensor and a status LED." Copper then runs the whole job:



- **Plan.** Split the requirement into blocks (power, microcontroller, sensor, LED) and the nets between them.
- **Parts.** Choose real components with their KiCad symbols and footprints.
- **Schematic.** Write it as code and generate the netlist.

- **ERC.** KiCad's electrical rules check. On failure, read the errors and fix the schematic.
- **Place and route.** Place the parts on a board outline and auto-route the copper tracks.
- **DRC.** KiCad's design rules check. On failure, fix placement or routing and check again.
- **Deliver.** Gerber files a factory such as JLCPCB accepts, a parts list (BOM) and a 3D render.

The checks are objective: KiCad says pass or fail, so the agent proves its work instead of claiming it.

4. Why this is high risk, high reward

	Risk	Reward
Task size	Many steps; any of them can break a run	Exactly what the 30% "agent" criterion asks for: a substantial task, start to finish
Verification	ERC and DRC are strict and will fail often	Pass/fail is objective, so failures become real neatlogs traces and real fixes
Placement	Automatic part placement is the hardest step	A routed board in 3D is a memorable demo moment
Originality	Few existing examples to learn from	Hardware agents are rare, which helps the originality score
Tool fit	KiCad automation is less familiar to AI agents	Every tool call and checker result is traceable: the neatlogs story

How we keep the risk survivable

If by...	this is not solid...	we ship...
Saturday night	schematic generation	the self-repair loop on schematics alone (still end to end, still verified)
Sunday evening	automatic placement	requirement → verified schematic → BOM → netlist; layout shown as work in progress
Monday 3 PM	anything new	freeze; only the video, README and evidence after this

Boards stay small (5–15 parts) on purpose. We measure and report the real pass rate, whatever it is.

5. How the required tools fit

Tool	Role in Copper	Evidence
neatlogs	Traces every step: model calls, KiCad tool runs, every ERC/DRC failure and its fix. The supervisor reads these traces over MCP to diagnose failures.	Trace links; a failed check and what the agent did next; before/after pass rates
Entire	Every self-improvement to the agent is a commit made by Claude Code, captured as a checkpoint with its reasoning.	Checkpoint links and IDs (already working today)
cfo.ai	Prices Copper per board design against an engineer's time, using our measured cost per design.	Exported business plan (a first model already exists)

6. What we already have (built and measured today)

- neatlogs tracing with per-run trace IDs, and reading traces back over the neatlogs MCP server.
- A self-repair supervisor: reads failing traces, writes a diagnosis, patches the agent with Claude Code, re-runs, keeps or reverts. On a code-fixing agent it found a real flaw (whole-file rewrites deleting code the agent had not read) and the pass rate went from 27/28 to 28/28.

- Entire capturing those headless Claude Code fixes as checkpoints; a cfo.ai account and first model; the public repo, team access and a first #neatHack post.
- KiCad 10 installed on the lead's machine (Java still needed for the auto-router).

Only the agent's tools and the test set change: invoicing bugs become small test boards.

7. Plan to submission (IST)

When	Goal
Sat evening	KiCad automation spike: generate a schematic from code, run ERC and export a netlist from Python.
Sat night	Agent: requirement → parts → schematic → ERC loop, traced in neatlogs; 5 test boards.
Sun day	Placement, auto-routing and the DRC loop; Gerbers and 3D render; self-repair rounds on failures.
Sun night	Freeze the agent; record the demo; cfo.ai plan with measured cost per design.
Mon by 3 PM	Video (3 min or less), README with evidence; submit on X from 7:00 PM.

Demo moment

Type the requirement. Watch the parts and schematic appear. DRC fails once; the agent reads the error and fixes it. The routed board appears in 3D next to its Gerber files.